

MakeLife: Data Flow Architecture

The following diagram and breakdown illustrate how information flows continuously between the Web Browser (Frontend), the Node.js Server (Backend), and the MongoDB Database.

Overview

1. **Frontend (React):** Handles user interactions and input forms.
2. **Backend (Express):** Receives HTTP requests, cleans data, and applies business logic.
3. **Database (MongoDB):** Safely stores the organized records.

Data Flow Diagram

```
graph TD
    %% Define User Classes
    PublicUser((Public User))
    AdminUser((Admin Staff))

    %% Define Frontend Components
    subgraph Frontend [React Frontend Window]
        DonationForms[Donation Forms]
        VolForms[Volunteer Forms]
        AdminDashboard[Admin Dashboard]
    end

    %% Define Backend API Routes
    subgraph Backend [Node.js / Express Backend]
        route_Donation[/api/donations/]
        route_Goods[/api/goods-donations/]
        route_Vol[/api/volunteers/]
        route_General[/api/... other routes]
        DataScrubber[Payload Scrubber]
    end

    %% Define Database Collections
    subgraph DB [MongoDB Database]
        col_Donations[(Donations Collection)]
        col_Goods[(Goods Collection)]
        col_Vols[(Volunteers Collection)]
        col_Other[(Other Collections)]
    end

    %% Define Flows from Public User
    PublicUser -->|Enters details & Clicks Submit| DonationForms
    PublicUser -->|Fills occupation/availability| VolForms

    %% Define Flows to Backend
    DonationForms -->|POST / JSON Payload| route_Donation
    DonationForms -->|POST / Extensive Item Details| route_Goods
    VolForms -->|POST / Application Details| route_Vol
```

```

%% Internal Backend Logic
route_Goods -->|Passes Payload| DataScrubber
DataScrubber -->|Removes Empty Strings| col_Goods

route_Donation -->|Save Direct| col_Donations
route_Vol -->|Save Direct| col_Vols

%% Admin Flows
AdminUser -->|Logs into UI| AdminDashboard
AdminDashboard -->|GET requests| route_General
AdminDashboard -->|GET requests| route_Goods
route_Goods -->|Fetch JSON array| col_Goods
col_Goods -->|Return JSON array| route_Goods
route_Goods -->|Send data back| AdminDashboard
AdminDashboard -->|Renders Table Rows| AdminUser

%% Admin Update Status
AdminDashboard -->|PATCH / status='approved'| route_Goods
route_Goods -->|Update Document Status| col_Goods

```

Step-by-Step Security & Cleansing Flow

- **Step 1 (Ingestion):** A user submits a lengthy form (e.g., Goods Donation specifying Book topics, Clothing items, quantities).
- **Step 2 (Cleansing):** The payload arrives at the Backend API. To prevent clutter in the database, the backend actively rips out any variable passed as "" (empty string) before parsing.
- **Step 3 (Storage):** The database securely writes the document. Due to Mongoose dynamic schemas, unselected categories simply vanish from the final MongoDB document, preventing "null" bloat.
- **Step 4 (Retrieval):** The Admin Dashboard requests the data. The backend fetches it from MongoDB and ships it natively as JSON array directly to the Admin UI for sorting and filtering.